

Cloud Provider Comparative Matrix for SREs

Expert Panel Context

Platform Engineer: These differences aren't academic. They affect MTTR, observability quality, and failure modes. A 5-minute incident in AWS might be 15 minutes in Azure due to metrics lag.

Incident Commander: Know your cloud's failure modes. AWS AZ failures behave differently than GCP zone failures. This determines your response strategy.

Staff SRE: This is hiring-critical. Interviewers ask: "How would you architect this in AWS vs GCP?" If you only know one cloud, you're limiting your opportunities.

Core Differences at a Glance

Aspect	AWS	GCP	Azure
Metrics Granularity	1-minute (standard), 1-second (detailed, \$\$)	1-minute (all metrics)	1-minute (standard)
Metrics Delay	1-3 minutes	1-2 minutes	2-5 minutes (higher lag)
Free Metrics Retention	15 months	6 weeks	93 days
Load Balancer Type	Multiple (ALB, NLB, CLB)	Global	Application Gateway, Load Balancer
Health Check Behavior	Instance-level	Instance + endpoint	Instance-level
Regional Isolation	Strong (AZs are separate facilities)	Strong (zones independent)	Variable (some regions share infra)

Aspect	AWS	GCP	Azure
Managed DB Failover	RDS: 60-120s	Cloud SQL: 30-60s	Azure SQL: 30-60s
Default Timeouts	Varies by service	Shorter defaults	Longer defaults
IAM Model	IAM (complex, powerful)	IAM (simpler, scoped)	RBAC + IAM (hybrid)

1. Observability & Metrics

AWS CloudWatch

Strengths:

- Deep integration with all AWS services
- Custom metrics easily added
- Logs Insights powerful query language
- 15-month retention

Weaknesses:

- 1-minute granularity (standard), requires \$\$\$ for 1-second
- 1-3 minute delay in metric availability
- Query syntax complex
- Per-metric costs can be high

SRE Impact:

Incident Detection:

- Alert fires 1-3 minutes after threshold breach
- During rapid failure, this is significant

Cost:

- 10,000 custom metrics = ~\$30/month
- 1-second granularity: 10x cost

PromQL Equivalent in CloudWatch:

```
# Request rate
AWS: Statistics=SUM, Period=60
PromQL: rate(http_requests_total[1m])

# Error rate percentage
AWS: (SUM(Errors) / SUM(Requests)) * 100
PromQL: (rate(errors[5m]) / rate(requests[5m])) * 100

# P99 latency
AWS: Statistics=p99
PromQL: histogram_quantile(0.99, rate(latency_bucket[5m]))
```

Best Practices:

- Use CloudWatch Agent for custom metrics
 - Set up CloudWatch Logs Insights for structured log queries
 - Export to Prometheus for complex queries
 - Use CloudWatch Alarms with SNS for alerting
-

GCP Cloud Monitoring (formerly Stackdriver)

Strengths:

- All metrics at 1-minute granularity by default
- Faster metric availability (1-2 min)
- Cleaner UI
- Better integration with open-source tools (Prometheus export)

Weaknesses:

- Only 6 weeks retention (short!)
- Fewer pre-built dashboards
- Query language less mature

SRE Impact:

Incident Detection:

- Slightly faster than AWS (1-2 min delay)

Retention:

- Must export to long-term storage (BigQuery, Prometheus)

Cost:

- First 150MB/month free
- After: \$0.258/MB

Monitoring Query Language (MQL) Examples:

```
# Request rate
fetch gce_instance
| metric 'compute.googleapis.com/instance/network/received_bytes_count'
| group_by 1m, [value_received_bytes_count_mean: mean(value_received_bytes_count)]

# Error rate
fetch cloud_run_revision
| { metric 'run.googleapis.com/request_count'
  | filter (metric.response_code_class == '5xx')
  ; metric 'run.googleapis.com/request_count' }
| ratio
```

Best Practices:

- Export metrics to BigQuery for long-term analysis
 - Use Prometheus for complex alerting
 - Leverage GCP's OpenTelemetry support
 - Set up log-based metrics for custom events
-

Azure Monitor

Strengths:

- Unified platform (metrics + logs + traces)
- Application Insights for APM
- Good Windows ecosystem integration
- Long retention (93 days free)

Weaknesses:

- 2-5 minute metric delay (worst of three)
- Query syntax (KQL) has learning curve
- Slower alert evaluation

SRE Impact:

```
Incident Detection:
- Slowest of three clouds (2-5 min delay)
- Critical for fast-moving incidents
```

Cost:

- First 5GB logs/month free
- After: \$2.76/GB

Kusto Query Language (KQL) Examples:

```
// Request rate
requests
| where timestamp > ago(5m)
| summarize count() by bin(timestamp, 1m)

// Error rate
requests
| where timestamp > ago(5m)
| summarize total=count(), errors=countif(success == false)
| extend error_rate = (errors * 100.0) / total

// P99 latency
requests
| where timestamp > ago(5m)
| summarize percentile(duration, 99)
```

Best Practices:

- Use Application Insights for distributed tracing
 - Set up Action Groups for alert routing
 - Export to Azure Data Explorer for complex analytics
 - Compensate for metric delay with shorter alert evaluation windows
-

2. Load Balancer Health Checks

AWS Elastic Load Balancer

Types:

- **ALB** (Application Load Balancer): L7, HTTP/HTTPS
- **NLB** (Network Load Balancer): L4, TCP/UDP
- **CLB** (Classic Load Balancer): Legacy

Health Check Behavior:

```
# ALB Health Check
HealthCheckPath: /health
HealthCheckIntervalSeconds: 30
HealthyThresholdCount: 2
UnhealthyThresholdCount: 2
HealthCheckTimeoutSeconds: 5

Time to mark unhealthy: 60 seconds (2 checks × 30s interval)
Time to mark healthy: 60 seconds
```

Failure Mode:

```
Scenario: Instance becomes unhealthy
t=0:      Health check fails
t=30:     Health check fails again
t=30:     Instance marked unhealthy, removed from rotation
t=30:     New connections stop routing to instance
t=30+:    Existing connections drain (if connection draining enabled)
```

SRE Implications:

- Minimum 30-60s to detect failure
- Connection draining adds time (can be disabled for fast removal)
- Failed health checks don't appear in CloudWatch by default (must enable)

Best Practices:

```
# Good health check endpoint
@app.route('/health')
def health():
    # Check critical dependencies
    if not db.is_connected():
        return "Unhealthy", 503
    if not cache.is_connected():
        return "Degraded", 200 # Still serve traffic
    return "Healthy", 200

# Time budget: Must respond in < 5s
```

GCP Load Balancer

Types:

- **Global HTTP(S) Load Balancer:** L7, anycast IP
- **Regional Load Balancer:** L4/L7
- **Internal Load Balancer:** VPC-internal

Health Check Behavior:

```
# Global LB Health Check
checkIntervalSec: 10
timeoutSec: 5
healthyThreshold: 2
unhealthyThreshold: 2

Time to mark unhealthy: 20 seconds (2 checks × 10s interval)
Time to mark healthy: 20 seconds
```

Failure Mode:

```
Scenario: Instance becomes unhealthy
t=0:      Health check fails
t=10:     Health check fails again
t=10:     Instance marked unhealthy
t=10:     New requests stop routing (existing requests complete)
t=10:     Backend drained (faster than AWS)
```

SRE Implications:

- Faster detection (10s vs 30s in AWS)
- Global load balancer can route around regional failures
- Health check logs available in Cloud Logging

Best Practices:

- Use `/healthz` convention (Kubernetes standard)
 - Return 200 for healthy, 503 for unhealthy
 - Check downstream dependencies in health endpoint
 - Monitor health check success rate as SLI
-

Azure Load Balancer

Types:

- **Application Gateway:** L7, WAF capabilities
- **Load Balancer:** L4, regional
- **Front Door:** Global, L7

Health Check Behavior:

```
# Application Gateway Health Check
Interval: 30 seconds
Timeout: 30 seconds
UnhealthyThreshold: 3

Time to mark unhealthy: 90 seconds (3 checks × 30s)
Time to mark healthy: 30 seconds
```

Failure Mode:

```
Scenario: Instance becomes unhealthy
t=0:      Health check fails
t=30:     Health check fails again
t=60:     Health check fails third time
t=90:     Instance marked unhealthy
t=90:     Traffic stops routing
```

SRE Implications:

- Slowest failure detection (90s)
- Can be faster with Load Balancer (not Application Gateway)
- Health probe logs in Azure Monitor

Best Practices:

- Use Azure Load Balancer for faster detection (not Application Gateway)
 - Monitor probe success rate in Azure Monitor
 - Implement graceful degradation (return 200 even if degraded, let app decide)
-

3. Managed Database Failure Modes

AWS RDS

Failover Time:

- Single-AZ: No automatic failover
- Multi-AZ: 60-120 seconds

Failover Process:

1. Primary failure detected (30-60s)
2. DNS CNAME updated to point to replica (30s)
3. Replica promoted to primary (30s)
4. Application connections break, retry, reconnect (0-30s)

Total: 60-120 seconds of downtime

Connection Behavior:

```
# Connection handling during failover
try:
    result = db.execute(query)
except OperationalError:
    # Connection broken during failover
    # Retry with new connection
    db = reconnect()
    result = db.execute(query)
```

Observable Signals:

```
CloudWatch Metrics:
- DatabaseConnections (drops to 0, then recovers)
- ReadLatency (spikes during promotion)
- ReplicaLag (increases, then resets)
```

SRE Implications:

- Application must handle connection failures
- Connection pools must detect and replace dead connections
- Some transactions may be lost if in-flight during failover
- Read replicas may lag by seconds (check ReplicaLag metric)

Best Practices:

- Set connect_timeout and read_timeout in connection string
 - Use connection pools with health checks
 - Implement retry logic with exponential backoff
 - Monitor ReplicaLag (alert if > 10s)
-

GCP Cloud SQL

Failover Time:

- HA Configuration: 30-60 seconds

Failover Process:

1. Primary failure detected (10-20s)
2. Standby promoted automatically (10-20s)
3. IP address updated (instant, uses internal LB)
4. Applications reconnect (10-20s)

Total: 30-60 seconds

Connection Behavior:

```
# GCP's Cloud SQL Proxy handles reconnection
# Applications see brief connection failure, then automatic recovery
```

```
# With Proxy:
```

```
db = sqlalchemy.create_engine('postgresql+pg8000://user:pass@dbname?unix_socket=/cloudsql/instance-name')
```

```
# Proxy handles failover transparently
```

Observable Signals:

Cloud Monitoring Metrics:

- database/cpu/utilization (drops during failover)
- database/network/connections (drops, recovers)
- database/replication/replica_lag (spikes, recovers)

SRE Implications:

- Faster than AWS (30-60s vs 60-120s)

- Cloud SQL Proxy simplifies connection handling
- Built-in connection pooling in proxy

Best Practices:

- Use Cloud SQL Proxy for automatic reconnection
 - Set aggressive connection timeouts (10-30s)
 - Monitor replica_lag metric
 - Test failover regularly (GCP makes it easy)
-

Azure SQL Database

Failover Time:

- Single database: 30-60 seconds
- Elastic pool: Similar

Failover Process:

1. Primary failure detected (10-20s)
2. Automatic failover to replica (10-20s)
3. TDS connection redirect (10-20s)
4. Application reconnects (0-10s)

Total: 30-60 seconds

Connection Behavior:

```
// Azure SQL connection with retry policy
SqlConnectionStringBuilder builder = new SqlConnectionStringBuilder();
builder.ConnectionString = "Server=tcp:myserver.database.windows.net,...";
builder.ConnectRetryCount = 3;
builder.ConnectRetryInterval = 10;

using (SqlConnection connection = new SqlConnection(builder.ConnectionString))
{
    connection.Open();
    // Automatic retry on transient failures
}
```

Observable Signals:

Azure Monitor Metrics:

- connection_failed (spikes during failover)
- deadlock (may increase briefly)
- cpu_percent (drops, recovers)

SRE Implications:

- Similar performance to GCP
- Built-in retry logic in SDKs
- Automatic tuning available

Best Practices:

- Use connection pooling with health checks
 - Enable automatic tuning (query performance)
 - Set ConnectRetryCount in connection string
 - Monitor connection_failed metric
-

4. Regional Failure Isolation

AWS Availability Zones

Architecture:

Region (us-east-1):

- ├─ AZ-a (separate physical datacenter)
- ├─ AZ-b (separate physical datacenter)
- └─ AZ-c (separate physical datacenter)

Isolation:

- Separate power
- Separate cooling
- Separate networking (mostly)
- <2ms latency between AZs

Failure Behavior:

Scenario: AZ-a fails (power loss)

Impact:

- Instances in AZ-a: Unreachable

- Load balancer: Stops routing to AZ-a
- RDS Multi-AZ: Fails over to AZ-b
- EBS volumes in AZ-a: Unavailable

Detection time: 30-60s (load balancer health checks)

Historical Failures:

- Dec 2021: us-east-1 AZ failure (3 hours)
- Affected: EC2, RDS, EBS in one AZ
- Multi-AZ deployments continued operating

SRE Design:

Minimum HA Architecture:

- Deploy across 2+ AZs
- Load balancer across AZs
- RDS Multi-AZ enabled
- Monitor per-AZ metrics

Cost:

- 2 AZs: 2x instances (minimum HA)
- 3 AZs: 3x instances (N+2 redundancy)

GCP Zones

Architecture:

Region (us-central1):

- ├─ Zone-a
- ├─ Zone-b
- ├─ Zone-c
- └─ Zone-f (some regions have 4+ zones)

Isolation:

- Separate physical locations
- Independent failure domains
- <1ms latency within region

Failure Behavior:

Scenario: Zone-a fails

Impact:

- Instances in zone-a: Unreachable
- Global load balancer: Stops routing to zone-a (10-20s)
- Cloud SQL: Fails over to zone-b (30-60s)
- Persistent disks in zone-a: Unavailable

Historical Failures:

- Nov 2023: europe-west9 zone failure (2 hours)
- Affected: Compute, storage in one zone
- Multi-zone deployments unaffected

SRE Design:

Minimum HA Architecture:

- Deploy across 2+ zones
- Use global load balancer (auto-routes around failures)
- Cloud SQL HA configuration
- Regional persistent disks (accessible across zones)

Cost:

- Similar to AWS (2x-3x instances)

Azure Availability Zones

Architecture:

Region (East US):

- ├─ Zone 1
- ├─ Zone 2
- └─ Zone 3

Note: Not all regions have zones

- Older regions: No zone support
- Newer regions: 3 zones

Failure Behavior:

Scenario: Zone 1 fails

Impact:

- VMs in Zone 1: Unreachable
- Load balancer: Stops routing to Zone 1
- Azure SQL: Fails over (30-60s)
- Managed disks: Unavailable (zone-local)

Important Difference:

- Not all regions support zones
- Some "regions" actually share infrastructure
- Must verify zone support before deployment

SRE Design:

Minimum HA Architecture:

- Check region for zone support
- If zones available: Deploy across 2+ zones
- If no zones: Use region pairs (cross-region DR)

Cost:

- Zone-redundant: 2x-3x cost
- Region-pair: Higher latency (cross-region)

5. Incident Response Differences

AWS-Specific Failure Modes

1. ELB Scaling Lag

Problem: ELB scales gradually

Symptom: Sudden traffic spike → 503 errors

Time: 1-7 minutes to scale

Mitigation:

- Pre-warm ELB (contact AWS support)
- Use NLB (faster scaling)

2. RDS Storage Full

Problem: EBS volume full → database read-only
Symptom: Write failures, app errors
Detection: FreeStorageSpace metric

Mitigation:

- Enable storage autoscaling
- Alert at 85% usage

3. Lambda Cold Starts

Problem: First invocation after idle → 1-10s latency
Symptom: P99 latency spike
Detection: Duration metric

Mitigation:

- Provisioned concurrency (\$\$\$)
- Keep-alive pings

GCP-Specific Failure Modes

1. Quota Exhaustion

Problem: Hit project quota (CPU, IP addresses, etc.)
Symptom: Auto-scaling fails, instances don't start
Detection: Quota usage metric

Mitigation:

- Request quota increase
- Monitor quota usage (alert at 80%)

2. Global Load Balancer Misconfiguration

Problem: Backend service misconfigured
Symptom: 502 Bad Gateway globally
Detection: Requests metric

Mitigation:

- Always test backend health
- Gradual traffic shift

3. Cloud SQL Maintenance Windows

Problem: Automated maintenance → brief downtime

Symptom: Connection failures

Detection: Maintenance schedule

Mitigation:

- Schedule during low-traffic window
- Handle connection failures gracefully

Azure-Specific Failure Modes

1. VM Reboot for Host Maintenance

Problem: Azure reboots VM for host updates

Symptom: Instance unreachable for 5-10 minutes

Detection: ServiceHealth events

Mitigation:

- Use availability sets
- Deploy across zones
- Handle instance failures

2. Storage Account Throttling

Problem: Exceed storage account IOPS/bandwidth

Symptom: Slow disk I/O, app slowdown

Detection: Throttling metric

Mitigation:

- Use Premium Storage
- Distribute across multiple storage accounts

3. Slow Metric Propagation

Problem: 2-5 minute delay in metrics

Symptom: Late incident detection

Detection: Compare timestamp vs current time

Mitigation:

- Shorten alert evaluation windows
- Use application-level metrics (faster)

6. Cost Optimization Impact on Reliability

AWS Cost vs Reliability Tradeoffs

Single AZ:

- Cost: 1x
- Reliability: ~99%
- Failure mode: AZ down = service down

Multi-AZ:

- Cost: 2x
- Reliability: ~99.9%
- Failure mode: AZ down = brief failover (60s)

Multi-Region:

- Cost: 3-5x (cross-region data transfer)
- Reliability: ~99.95%
- Failure mode: Region down = traffic shifts (DNS/routing)

GCP Cost vs Reliability Tradeoffs

Single Zone:

- Cost: 1x
- Reliability: ~99%
- Failure mode: Zone down = service down

Multi-Zone:

- Cost: 2x
- Reliability: ~99.9%
- Failure mode: Zone down = failover (30s)

Multi-Region:

- Cost: 2-3x (better cross-region pricing than AWS)
- Reliability: ~99.95%
- Failure mode: Region down = global LB reroutes (instant)

Azure Cost vs Reliability Tradeoffs

Single Zone:

- Cost: 1x
- Reliability: ~99%
- Only in zone-supported regions

Availability Set (within one datacenter):

- Cost: 1x
- Reliability: ~99.5%
- Failure mode: Rack/host failures only

Availability Zones:

- Cost: 2x
- Reliability: ~99.9%
- Failure mode: Zone down = failover (30-60s)

Summary: When to Choose Which Cloud

Choose AWS When:

- Need widest service selection
- Complex IAM requirements
- Strong ecosystem (third-party integrations)
- Mature tooling and documentation

SRE Tradeoffs:

- Longer metric delays (1-3 min)
- More complex pricing
- Requires more tuning for optimal performance

Choose GCP When:

- Need fast metric propagation (1-2 min)
- Want simpler pricing model
- Prefer open-source tooling (Kubernetes, Prometheus)

- Global presence needed (global LB)

SRE Tradeoffs:

- Shorter metrics retention (6 weeks)
 - Fewer third-party integrations
 - Smaller service catalog
-

Choose Azure When:

- Microsoft ecosystem (Windows, .NET)
- Enterprise integration (Active Directory)
- Hybrid cloud requirements

SRE Tradeoffs:

- Longest metric delays (2-5 min)
 - Zone support varies by region
 - Learning curve for KQL
-

Interview Preparation: Cloud Comparison Questions

Question: "How would you design high-availability differently in AWS vs GCP?"

Answer:

"In AWS, I'd use Multi-AZ deployments with ALB and RDS Multi-AZ, expecting 60-120s failover. In GCP, I'd use multi-zone with global load balancer and Cloud SQL HA, expecting 30-60s failover. Key difference: GCP's global LB can route around zone failures globally, while AWS requires regional LBs. Both require 2x cost for basic HA."

Question: "Your CloudWatch metrics show everything healthy but users report issues. Why?"

Answer:

"CloudWatch has 1-3 minute propagation delay. The issue may be recent (< 3 min). Also, check:

- Per-endpoint metrics (aggregate may hide specific endpoint failures)
- Application-level metrics (more granular than CloudWatch)
- User geography (could be CDN/routing issue not visible in metrics)"

Question: "Compare observability in AWS vs GCP vs Azure."

Answer:

"Metric propagation: GCP fastest (1-2 min), AWS middle (1-3 min), Azure slowest (2-5 min). This

affects MTTR. Retention: AWS best (15 months), Azure medium (93 days), GCP worst (6 weeks). For production SRE work, I'd export all three to Prometheus for consistent interface and longer retention."